

Introducción a VHDL

Matías Javier Oliva
Pablo García

Implementación en FPGA

- Todo el hardware visto durante esta materia puede implementarse en dispositivos de lógica programable o en hardware específico ASIC.
- En particular las FPGAs son arreglos de celdas lógicas programables que se pueden usar para probar e implementar sistemas.
- Estos sistemas permiten aprovechar las ventajas del diseño específico de hardware, como la ejecución paralela de procesos.
- Aunque son más lentas que dispositivos ASIC permiten mucha más flexibilidad, y son un elemento clave en el prototipado de los sistemas que luego se implementarán en ellos.

Implementación en FPGA

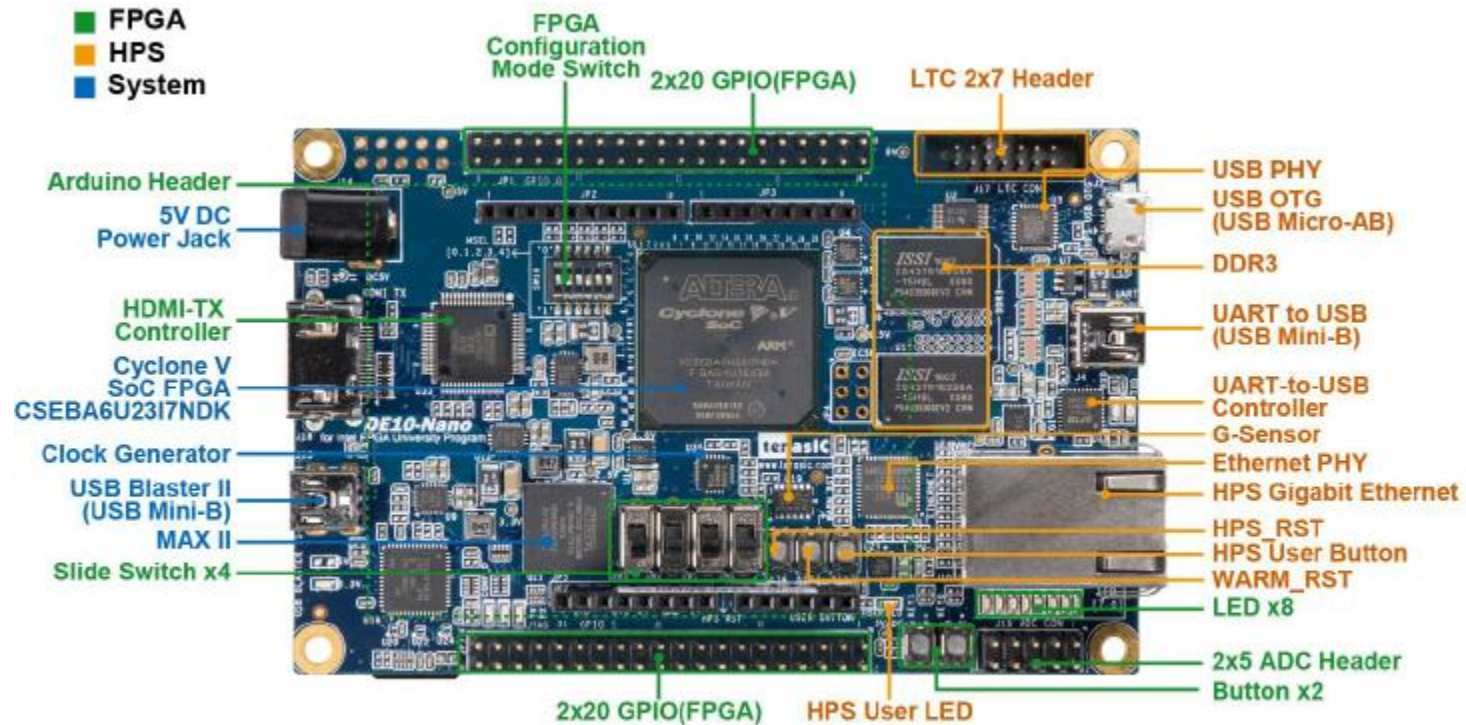


Figure 2-1 DE10-Nano development board (top view)

Primer ejemplo: Máquina de estados simple

 Device ×

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

Device family

Family: Cyclone V (E/GX/GT/SX/SE/ST)

Devices: All

Show in 'Available devices' list

Package: Any

Pin count: Any

Speed grade: Any

Name filter:

☒ Show advanced devices

Device and Pin Options...

Target device

☐ Auto device selected by the Fitter

☒ Specific device selected in 'Available devices' list

☐ Other: n/a

Available devices:

Name	Core Voltage	ALMs	User I/Os	GXB Channel PMA	GXB Channel PCS
5CSEBA6U23A7	1.1V	41910	314	0	0
5CSEBA6U23C6	1.1V	41910	314	0	0
5CSEBA6U23C7	1.1V	41910	314	0	0
5CSEBA6U23C7S	1.1V	41910	314	0	0
5CSEBA6U23C8	1.1V	41910	314	0	0
5CSEBA6U23C8S	1.1V	41910	314	0	0
5CSEBA6U23I7	1.1V	41910	314	0	0
5CSEBA6U23I7S	1.1V	41910	314	0	0
5CSEMA2U23A7 (Advanced)	1.1V	9430	314	0	0
5CSEMA2U23C6 (Advanced)	1.1V	9430	314	0	0
5CSEMA2U23C7 (Advanced)	1.1V	9430	314	0	0
5CSEMA2U23C8 (Advanced)	1.1V	9430	314	0	0
5CSEMA2U23I7 (Advanced)	1.1V	9430	314	0	0
5CSEMA4U23A7 (Advanced)	1.1V	15880	314	0	0

Migration Devices... 0 migration devices selected

[Buy Software](#) OK Cancel Help

Ahora sí resulta importante elegir correctamente el dispositivo apuntado

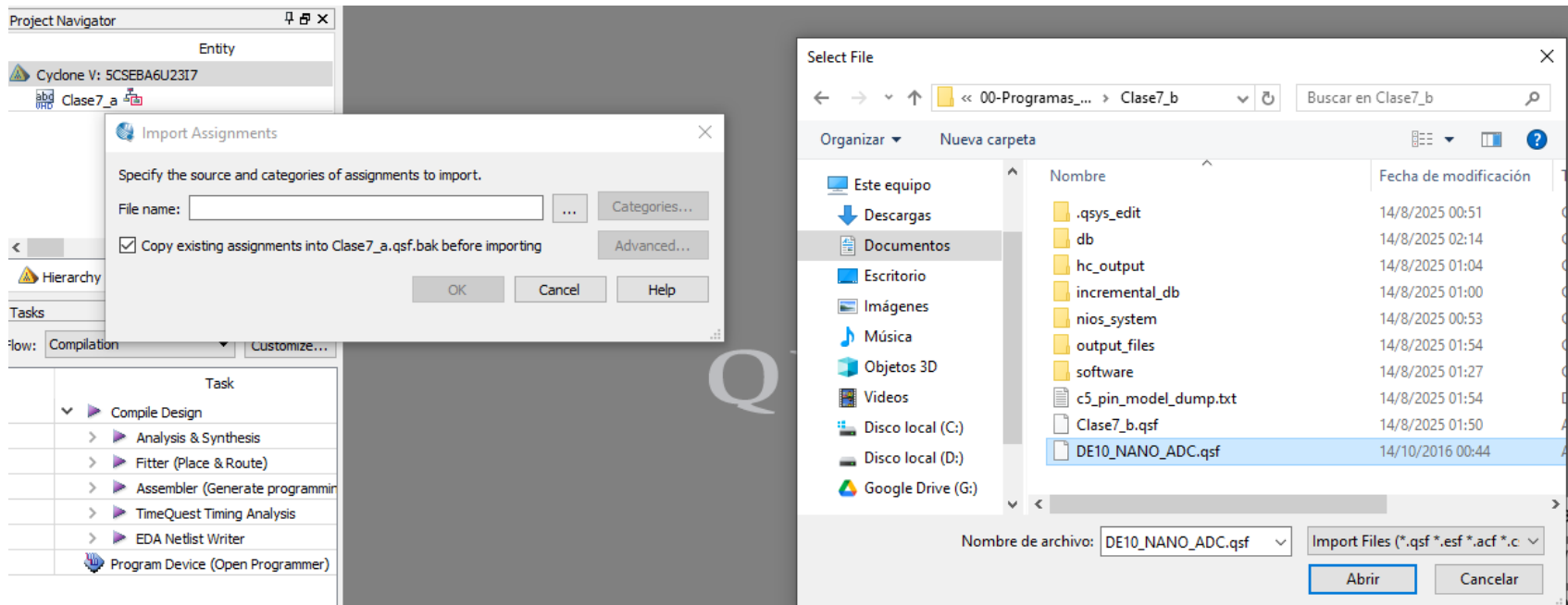
Primer ejemplo: Máquina de estados simple

- Cuando declaramos inputs y outputs en VHDL estos no están apuntando específicamente a ningún PIN físico de una placa de desarrollo. Para asociar los pines con los distintos periféricos presentes en una placa de desarrollo como la de10 nano Quartus nos provee varias herramientas.
- La más sencilla es contar con un archivo .qsf, que es un archivo de texto que describe como están mapeados los pines de la FPGA con los distintos periféricos:

```
set_location_assignment PIN_AH17 -to KEY[0]
set_location_assignment PIN_AH16 -to KEY[1]
set_location_assignment PIN_W15 -to LED[0]
set_location_assignment PIN_AA24 -to LED[1]
set_location_assignment PIN_V16 -to LED[2]
set_location_assignment PIN_V15 -to LED[3]
set_location_assignment PIN_AF26 -to LED[4]
set_location_assignment PIN_AE26 -to LED[5]
set_location_assignment PIN_Y16 -to LED[6]
set_location_assignment PIN_AA23 -to LED[7]
set_location_assignment PIN_Y24 -to SW[0]
set_location_assignment PIN_W24 -to SW[1]
set_location_assignment PIN_W21 -to SW[2]
set_location_assignment PIN_W20 -to SW[3]
```

Primer ejemplo: Máquina de estados simple

- Assignments -> Import assignments



- Con esta herramienta se importa directamente el .qsf de la placa a nuestro proyecto. Notar que para que esto tenga éxito los inputs y outputs de nuestro proyecto deben tener los nombres específicos descriptos en el .qsf

Primer ejemplo: Máquina de estados simple

- Assignments -> Pin planner

Pin Planner - C:/Users/mati9/Documents/00-Programas_curso_vhdl/Clase7_a/Clase7_a - Clase7_a

File Edit View Processing Tools Window Help

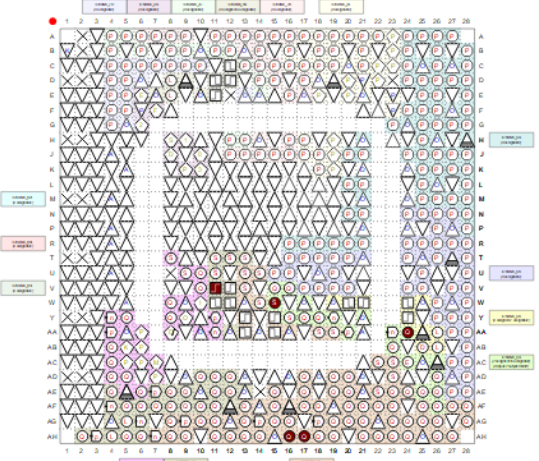
Report Report not available

Groups Report

Tasks

- Early Pin
 - Early
 - Run
 - Expo
 - Pin Finde
- Highlight
 - I/O E

Top View - Wire Bond
Cyclone V - 5CSEBA6U2317



Named: * Edit:

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate
clk	Input	PIN_V11	3B	B3B_N0	PIN_V11	2.5 V (default)		12mA (default)	
entrada	Input	PIN_AH17	4A	B4A_N0	PIN_AH17	2.5 V (default)		12mA (default)	
reset	Input	PIN_AH16	4A	B4A_N0	PIN_AH16	2.5 V (default)		12mA (default)	
salida[1]	Output	PIN_W15	5A	B5A_N0	PIN_W15	2.5 V (default)		12mA (default)	1 (default)
salida[0]	Output	PIN_AA24	5A	B5A_N0	PIN_AA24	2.5 V (default)		12mA (default)	1 (default)
<<new node>>									

- Otra herramienta posible es el Pin Planner que permite asociar manualmente los pines con los puertos de nuestro prouyecto. Los pines asociados a cada periférico pueden encontrarse en el manual de la placa.

Primer ejemplo: Máquina de estados simple

Named: *	Edit:									
Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	
clk	Input	PIN_V11	3B	B3B_N0	PIN_V11	2.5 V (default)		12mA (default)		
entrada	Input	PIN_AH17	4A	B4A_N0	PIN_AH17	2.5 V (default)		12mA (default)		
reset	Input	PIN_AH16	4A	B4A_N0	PIN_AH16	2.5 V (default)		12mA (default)		
salida[1]	Output	PIN_W15	5A	B5A_N0	PIN_W15	2.5 V (default)		12mA (default)	1 (default)	
salida[0]	Output	PIN_AA24	5A	B5A_N0	PIN_AA24	2.5 V (default)		12mA (default)	1 (default)	
<<new node>>										

- Como entrada vamos a usar el KEY(0) de la placa (activo en bajo)
- Como salidas vamos a usar los LED 0 y 1 de la placa.
- Nuestro sistema va a encender LED(0), cuando se apriete KEY(0) va a encender LED(1) y así sucesivamente

```

library ieee;
use ieee.std_logic_1164.all;

entity Clase7_a is
    port(
        clk      : in std_logic;
        entrada  : in std_logic;
        reset    : in std_logic;
        salida   : out std_logic_vector (1 downto 0)
    );
end entity;

```


Primer ejemplo: Máquina de estados simple

```

process (clk, reset)
begin
    if reset = '0' then
        state <= s0;
    elsif (rising_edge(clk)) then
        case state is
            when s0=>
                if entrada = '0' then
                    state <= s1;
                else
                    state <= s0;
                end if;
            when s1=>
                if entrada = '1' then
                    state <= s2;
                else
                    state <= s1;
                end if;
            when s2=>
                if entrada = '0' then
                    state <= s3;
                else
                    state <= s2;
                end if;
            when s3 =>
                if entrada = '1' then
                    state <= s0;
                else
                    state <= s3;
                end if;
        end case;
    end if;
end process;

```

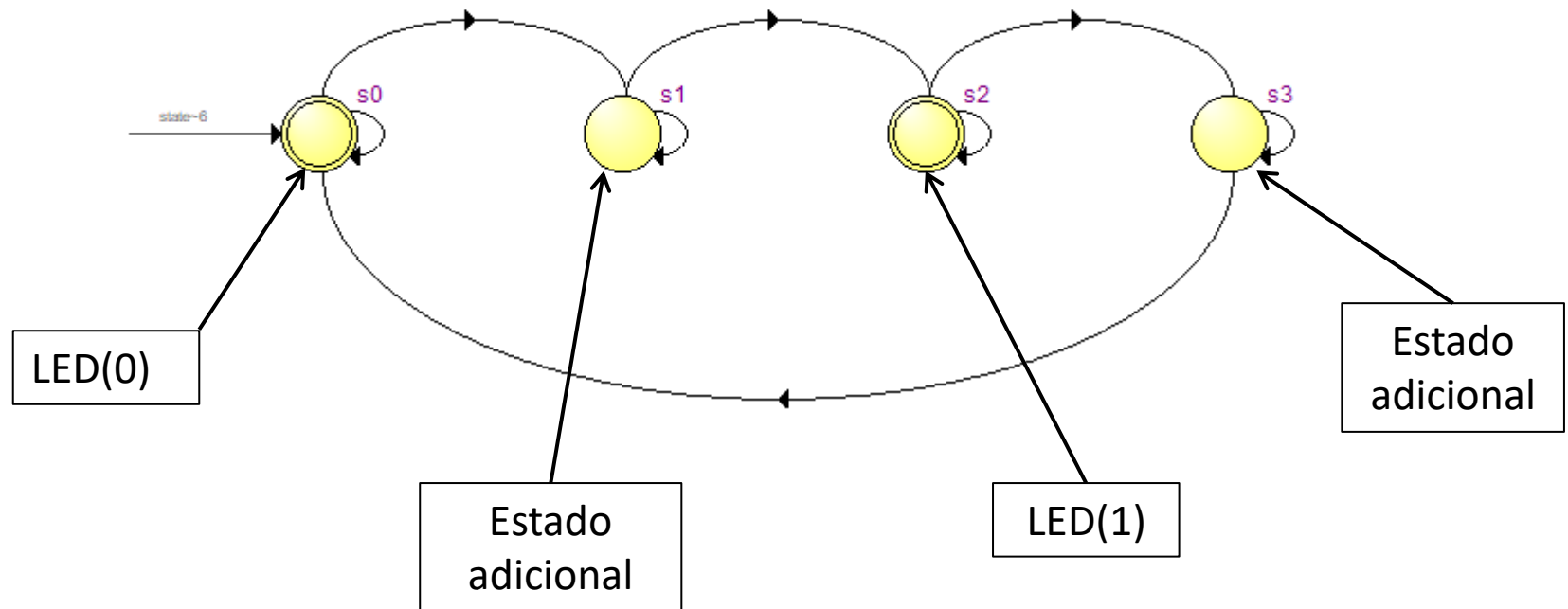
```

-- salida depends solely on the current state
process (state)
begin
    case state is
        when s0 =>
            salida <= "01";
        when s1 =>
            salida <= "00";
        when s2 =>
            salida <= "10";
        when s3 =>
            salida <= "00";
    end case;
end process;

```

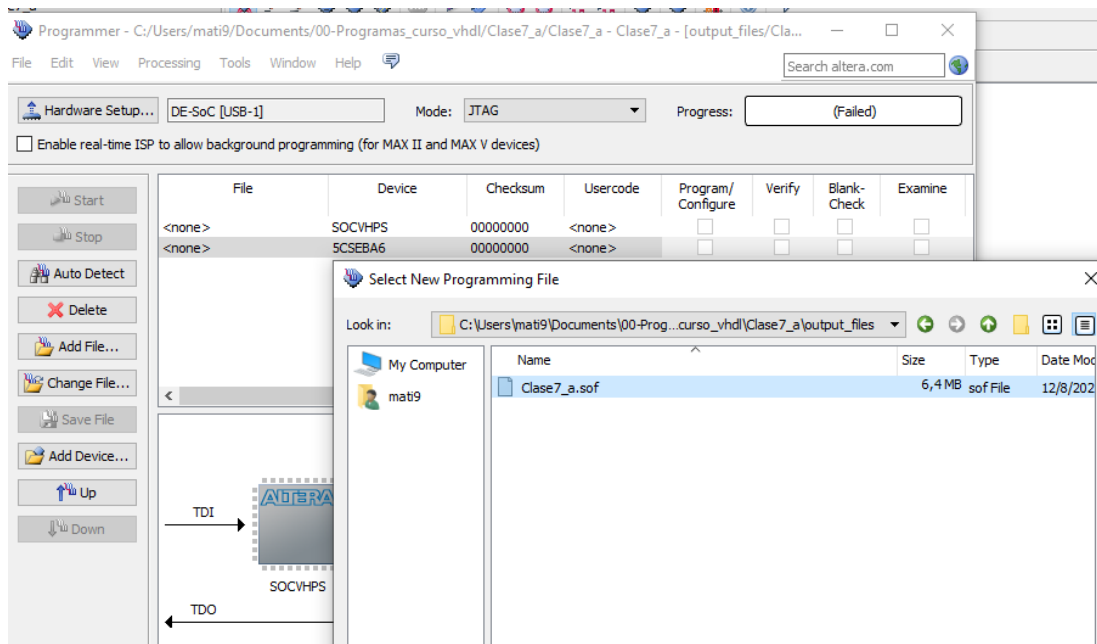
- El sistema emplea estados adicionales (s1 y s3) para solucionar el problema de que los KEY están apretados por muchos ciclos de reloj realmente.

Primer ejemplo: Máquina de estados simple



Primer ejemplo: Máquina de estados simple

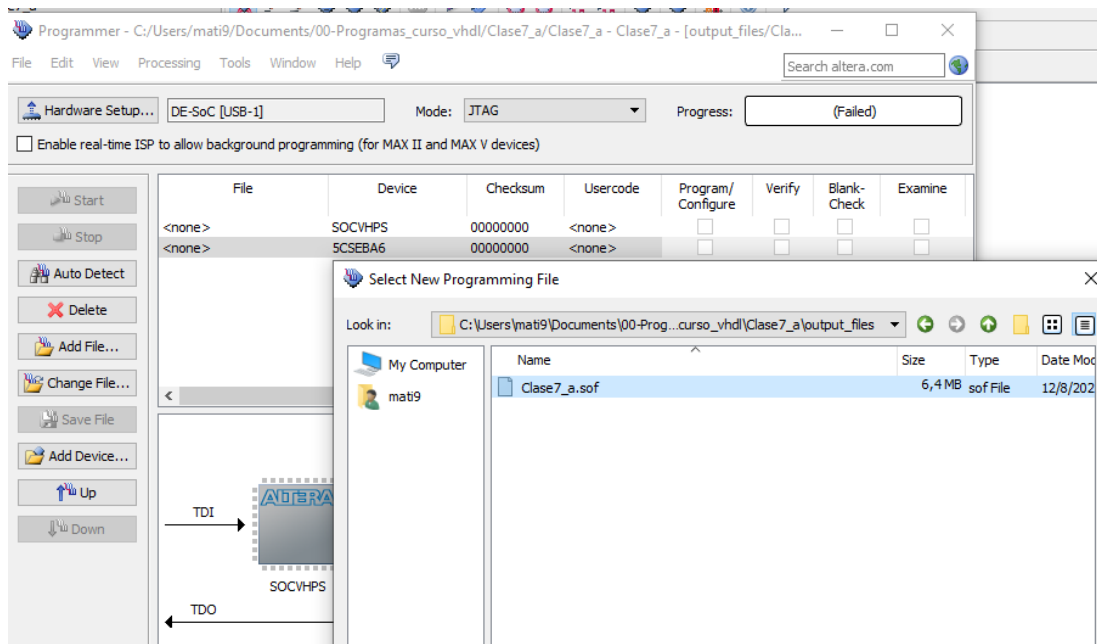
- Una vez compilado el proyecto se genera un archivo .sof que es el que se programará en la FPGA. Para “bajar” el proyecto a la FPGA se puede usar una herramienta de Quartus llamada “programmer”



- Se selecciona el .sof correspondiente y la FPGA objetivo en hardware setup (es necesario tener el driver adecuado)

Primer ejemplo: Máquina de estados simple

- Una vez compilado el proyecto se genera un archivo .sof que es el que se programará en la FPGA. Para “bajar” el proyecto a la FPGA se puede usar una herramienta de Quartus llamada “programmer”



- Se selecciona el .sof correspondiente y la FPGA objetivo en hardware setup (es necesario tener el driver adecuado)
- Una vez programado el hardware se llena la barra de progreso y debería arrancar a funcionar nuestra secuencia de LEDS

Sistemas más complejos

- Con estrategias como esta se pueden implementar todos los ejemplos vistos durante este curso en FPGAs de Intel-Altera. Sin embargo hay una limitación importante: los medios para gestionar datos de entrada y salida se reducen a un par de KEYS, Switches y LEDs.
- Para aprovechar correctamente el procesamiento de bajo nivel, paralelismo y especificidad que uno puede lograr con Hardware es necesario una interfaz de usuario más dinámica, es decir alguna interfaz de alto nivel que permita ingresar y exportar datos.
- Existen varias formas de lograr esto. Durante este curso vamos a ver:
 - Microprocesadores implementados directamente en hardware.
 - Microprocesadores embebidos en el chip de la FPGA.

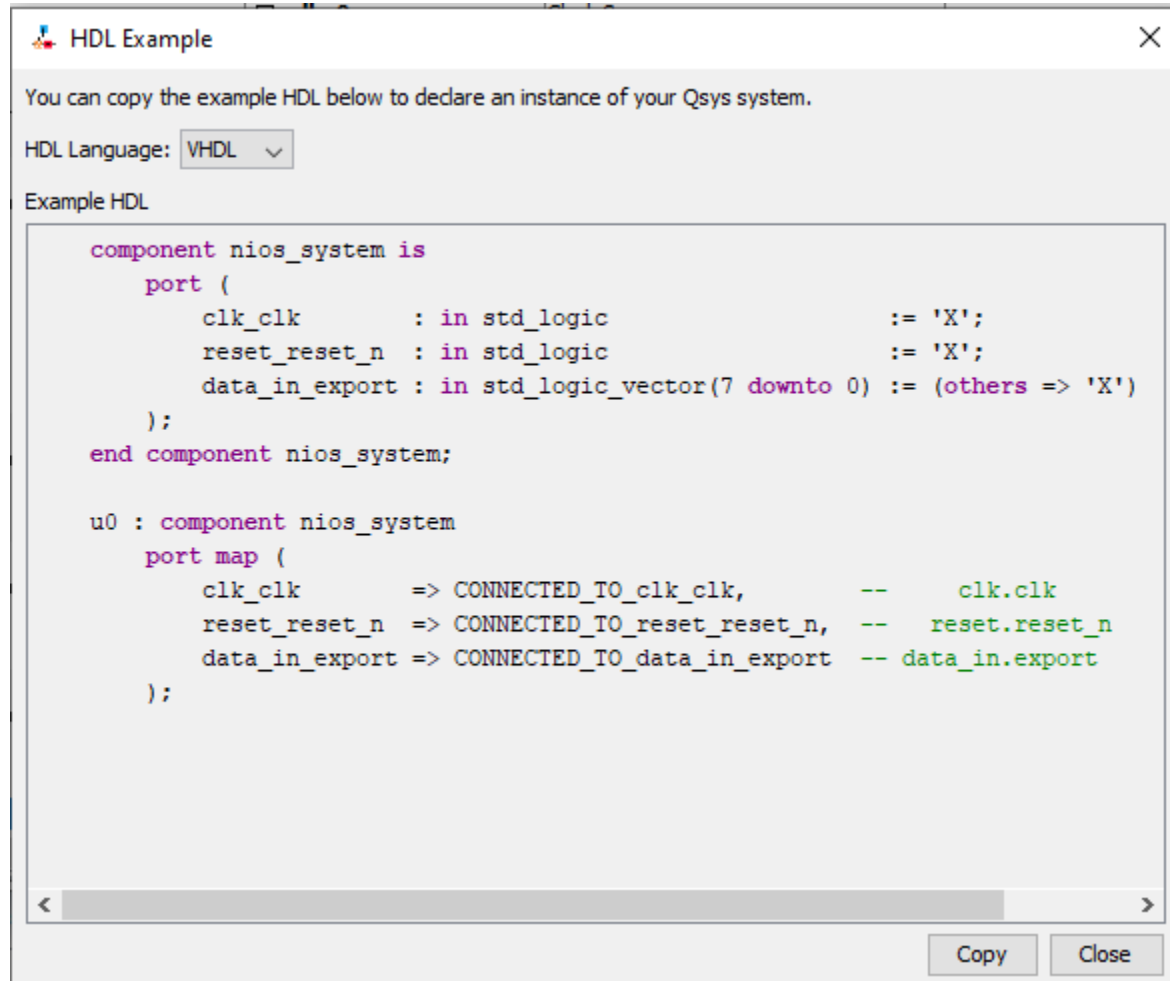
Procesador NIOS

- El procesador NIOS es un procesador que nos provee Intel-Altera que no es más que una IP que programa algunas de las celdas lógicas de la FPGA para implementar un procesador RISC.
- Mediante puertos de entrada y salida conectados a este procesador puede implementarse procesamiento directamente en hardware y gestionar los datos mediante el procesador.
- Para implementarlo aparte del procesador en sí se necesitan algunos bloques extra:
 - Módulo de memoria (recomendable 128 kbits)
 - Programador JTAG
- Todas estas funcionalidades pueden incorporarse a un sistema como un componente más, a través de la herramienta QSYS.

Procesador NIOS

System Contents							
Address Map							
Project Settings							
Use	Connections	Name	Description	Export	Clock	Base	
☒		clk_0	Clock Source	clk	<i>Double-click to export</i> <i>Double-click to export</i>	exported	
		clk_in	Clock Input	reset		clk_0	
		clk_in_reset	Reset Input				
		clk	Clock Output				
		clk_reset	Reset Output				
☒		nios2_qsys_0	Nios II Processor		clk_0		
		clk	Clock Input	<i>Double-click to export</i>	[clk]		
		reset_n	Reset Input	<i>Double-click to export</i>	[clk]		
		data_master	Avalon Memory Mapped Master	<i>Double-click to export</i>	[clk]		
		instruction_master	Avalon Memory Mapped Master	<i>Double-click to export</i>	[clk]		
		jtag_debug_module_r...	Reset Output	<i>Double-click to export</i>	[clk]		
		jtag_debug_module	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk]		
		custom_instruction_m...	Custom Instruction Master	<i>Double-click to export</i>	[clk]		
☒		program_memory	On-Chip Memory (RAM or ROM)		clk_0		
		clk1	Clock Input	<i>Double-click to export</i>	[clk1]		
		s1	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk1]		
		reset1	Reset Input	<i>Double-click to export</i>			
☒		jtag_uart_0	JTAG UART		clk_0		
		clk	Clock Input	<i>Double-click to export</i>	[clk]		
		reset	Reset Input	<i>Double-click to export</i>	[clk]		
		avalon_jtag_slave	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk]		
☒		pio_0	PIO (Parallel I/O)		clk_0		
		clk	Clock Input	<i>Double-click to export</i>	[clk]		
		reset	Reset Input	<i>Double-click to export</i>	[clk]		
		s1	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk]		
		external_connection	Conduit	data_in			

Procesador NIOS



HDL Example

You can copy the example HDL below to declare an instance of your Qsys system.

HDL Language:

Example HDL

```
component nios_system is
  port (
    clk_clk      : in std_logic           := 'X';
    reset_reset_n : in std_logic           := 'X';
    data_in_export : in std_logic_vector(7 downto 0) := (others => 'X')
  );
end component nios_system;

u0 : component nios_system
  port map (
    clk_clk      => CONNECTED_TO_clk_clk,      -- clk.clk
    reset_reset_n => CONNECTED_TO_reset_reset_n, -- reset.reset_n
    data_in_export => CONNECTED_TO_data_in_export -- data_in.export
  );
```

< >

Copy Close

Procesador NIOS

```

architecture arch of Clase7_b is
  signal clk : std_logic;

  component nios_system is
    port (
      clk_clk      : in std_logic      := 'X';           -- clk
      reset_reset_n : in std_logic      := 'X';           -- reset_n
      data_in_export : in std_logic_vector(7 downto 0) := (others => 'X') -- export
    );
  end component nios_system;

  signal data_in : std_logic_vector(7 downto 0);

begin

  clk <= FPGA_CLK1_50;
  data_in <= "0000" & sw;

  u0 : component nios_system
    port map (
      clk_clk      => clk,           -- clk.clk
      reset_reset_n => key(0),       -- reset.reset_n
      data_in_export => data_in      -- data_in.export
    );

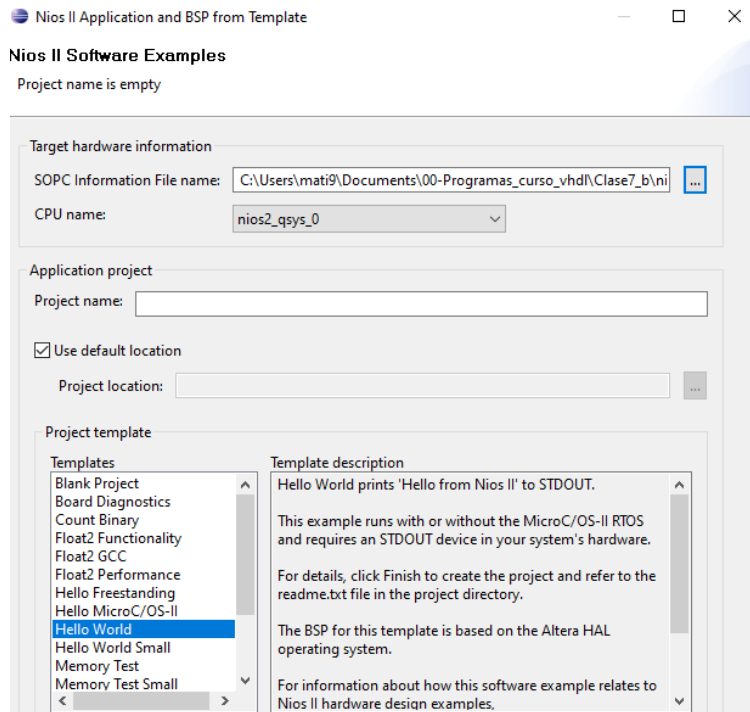
end arch;

```

Para que el sistema lo reconozca es necesario incorporarlo al proyecto mediante un archivo .qip

Procesador NIOS

- Una vez diseñado el HDL con su procesador NIOS puede emplearse la herramienta nios2 software build tools for Eclipse para generar la capa de aplicación del procesador.



- Una vez abierto el Eclipse se genera el board support package (file -> new -> Nios 2 application and BSP from template)

Procesador NIOS

- Con el BSP diseñado se genera el programa que corre el procesador. En este caso va imprimiendo los valores que recibe en el SW:

```
#include <stdio.h>
#include "system.h"

int main()
{
    printf("Hello from Nios II!\n");

    volatile int * pointer_to_pio = (volatile int *) PIO_0_BASE;

    while(1)
    {
        printf("%d", *pointer_to_pio);|
        getchar();

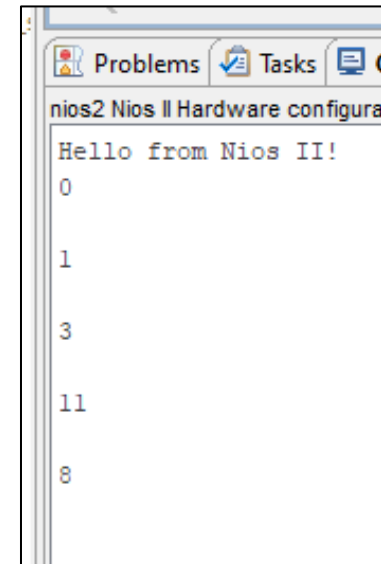
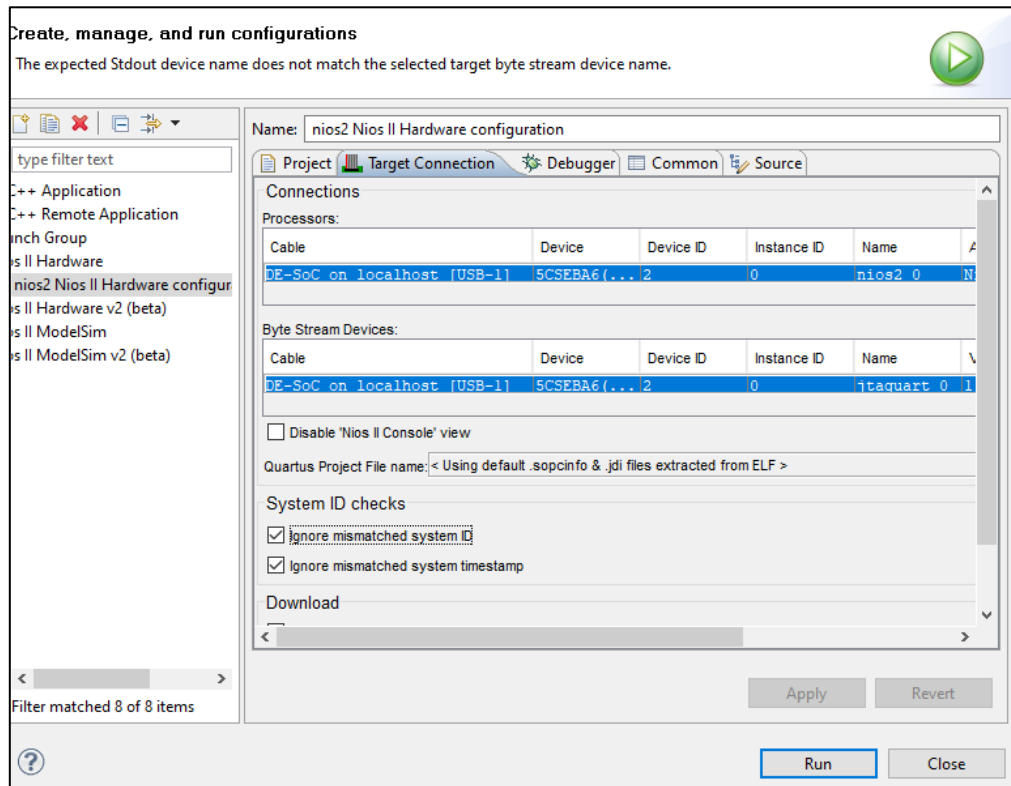
    }

    return 0;
}
```

Los addresses del paralel output/input usados en este ejemplo (definidos en QSYS) se incluyen en el header “system.h” generado por el BSP

Procesador NIOS

- Por último hay que configurar el “run” para que encuentre el procesador NIOS dentro de la FPGA (el HDL que lo implementa debe estar cargado en la placa usando el programmer)



Último ejemplo: ADC

- Para empezar a hacer cosas más “útiles” con un sistema como este está bueno incorporar un módulo ADC, que pueda ser gestionado por el procesador NIOS.
- Para esto aprovechamos un módulo que ya teníamos implementado en Verilog, que gestiona el ADC LTC 2308, presente en la de10nano.

```
module embedded_adc(  
  
    input clk,  
    input reset_n,  
    input enable,  
  
    input [31:0] fmuestreo,  
    input [3:0] sel_ch,  
  
    ////////// ADC //////////  
    output adc_cs_n,  
    output adc_sclk,  
    output adc_din,  
    input  adc_dout,  
  
    ///// SALIDAS //////////  
  
    output data_stream_valid_adc,  
    output [31:0] data_stream_adc  
  
);
```

- Es interesante notar que si bien este módulo está escrito en Verilog puede interfacearse directamente con un sistema en VHDL.
- Solo necesitamos proveerlo de estos puertos input y output que son pines físicamente disponibles en la placa, de una frecuencia de muestreo, una selección de canal y señales de clk y reset.

Último ejemplo: ADC

```

component embedded_adc
  port (
    clk      : in  std_logic;
    reset_n  : in  std_logic;
    enable    : in  std_logic;
    fmuestreo : in std_logic_vector(31 downto 0);
    sel_ch    : in std_logic_vector(3 downto 0);
    adc_cs_n  : out std_logic;
    adc_sclk  : out std_logic;
    adc_din   : out std_logic;
    adc_dout  : in  std_logic;
    data_stream_valid_adc : out std_logic;
    data_stream_adc       : out std_logic_vector(31 downto 0)
  );
end component;

```

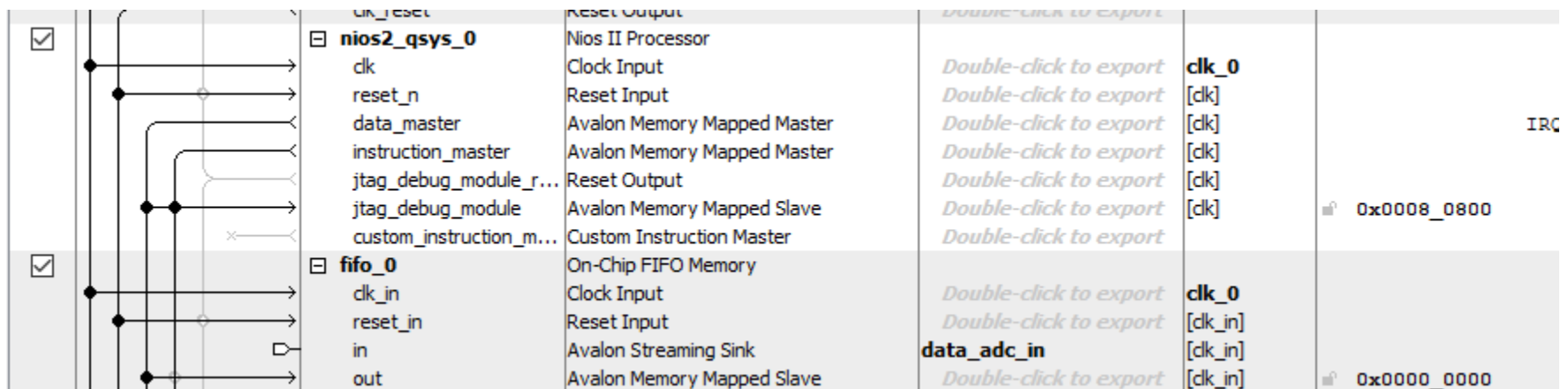
```

ul: embedded_adc
  port map (
    clk      => clk,
    reset_n => key(0),
    enable   => '1',
    fmuestreo => std_logic_vector(to_unsigned(1000, 32)),
    sel_ch   => "0001",
    adc_cs_n => ADC_CONVST,
    adc_sclk => ADC_SCK,
    adc_din  => ADC_SDI,
    adc_dout => ADC_SDO,
    data_stream_valid_adc => data_adc_valid,
    data_stream_adc       => data_adc
  );

```

Último ejemplo: ADC

- La interfaz de salida de este módulo es conocida como “Avalon Streaming”. La idea es que en cada ciclo de reloj en que hay una muestra válida se pone en alto una señal “data_valid”.
- Estas señales se pueden conectar directamente a IP de Altera que tenga esta interfaz de entrada.
- Podemos incorporar al sistema NIOS un FIFO que tenga esta interfaz de entrada, con una interfaz de salida que se conecte con el procesador.



Último ejemplo: ADC

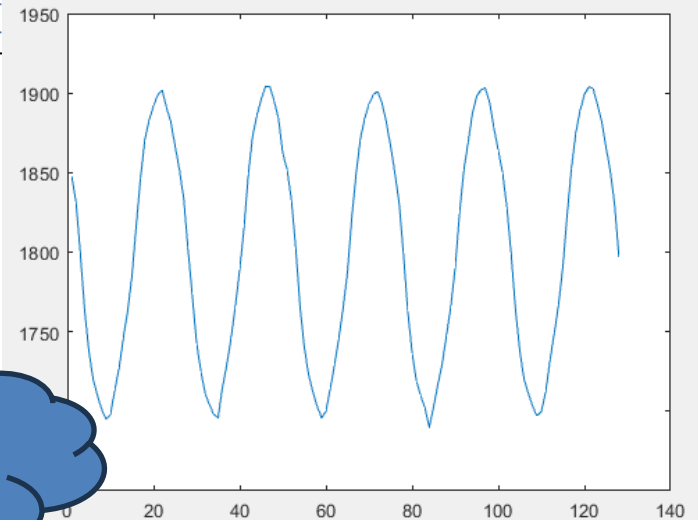
- Luego podemos leer los contenidos del FIFO directamente con un programa en el NIOS:

```
int i = 0;
for (i = 0; i<128;i++){
    printf("%d, ",*pointer_to_fifo);
}
getchar();
```

Programa

1848,	1832,	1800,	1764,	1738,	1720,	1710,	1701,	1695,	1698,	1713,	1727,	1746,	1763,	1785,	1817,	1847,	1871,	1883,
1892,	1899,	1902,	1891,	1882,	1867,	1852,	1834,	1802,	1773,	1743,	1725,	1711,	1704,	1698,	1696,	1715,	1729,	1746,
1766,	1789,	1815,	1849,	1873,	1886,	1896,	1904,	1904,	1895,	1884,	1862,	1852,	1833,	1803,	1766,	1740,	1723,	1712,
1703,	1696,	1700,	1714,	1729,	1746,	1765,	1787,	1821,	1849,	1871,	1884,	1893,	1899,	1901,	1894,	1882,	1867,	1850,
1831,	1799,	1763,	1737,	1719,	1710,	1702,	1690,	1703,	1717,	1730,	1748,	1766,	1790,	1824,	1851,	1869,	1887,	1898,
1902,	1903,	1894,	1878,	1864,	1850,	1828,	1799,	1764,	1738,	1720,	1711,	1700,	1697,	1698,	1699,	1698,	1699,	1698,
1791,	1824,	1853,	1875,	1889,	1899,	1904,	1903,	1893,	1882,	1866,	1851,	1	1950					

Salida de la consola



Datos graficados